AI-Assisted Coding

Assignment-13.5

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Name: S Srinidhi

HT No.:2303A51342

Batch:**20**

1. Refactoring – Removing Global Variables

• Task: Use AI to eliminate unnecessary global variables from the code

Prompt:

Refactor the following Python code to remove the global variable. Pass the variable as a function parameter to improve modularity and testability. Follow modern Python best practices and keep the output the same.

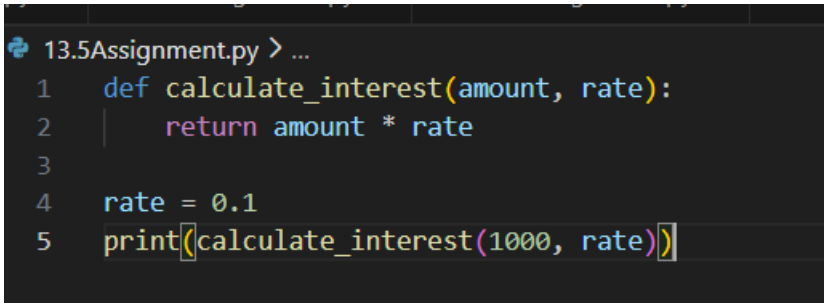
Legacy Code:
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))

Code:

```
def calculate_interest(amount, rate):
    return amount * rate

rate = 0.1

print(calculate_interest(1000, rate))
```



```
rive/Desktop/AI_Assistant_Lab/13.5Assignment.py
100.0
```

Observation:

In the legacy code, the interest rate was declared as a global variable and used directly inside the function. This created tight coupling and reduced the flexibility of the function. During refactoring, the global variable was removed and the rate value was passed as a parameter to the function. This improved modularity and allowed the function to work with different values easily. It also enhanced testability because the function behavior no longer depends on external variables.

2: (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

Prompt:

Refactor the following Python code to remove deeply nested if–else statements. Simplify the logic using elif or a cleaner structure while maintaining the same output. Focus on readability and maintainability.

Legacy Code:
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")

Code:

```
score = 78

if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
```

elif score >= 60:

print("Good")

else:

print("Needs Improvement")

```python
#task2
score = 78
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")
```

```
rive/Desktop/AI_Assistant_Lab/13.5Assignment.py
100.0
Very Good
```

Observation:

The original program used multiple nested if–else statements to evaluate the score conditions. This structure made the code harder to read and understand. The refactored version replaced the nested conditions with a clean if–elif–else structure. This flattened the logic and improved readability significantly. As a result, the code became easier to maintain and modify without affecting functionality.

3 .Refactoring Repeated File Handling Code)

• Task: Use AI to refactor repeated file open/read/close logic.

Prompt:

Refactor the following Python code to eliminate repeated file open/read/close logic. Use a reusable function and a context manager (with open) to follow the DRY principle and improve code readability.

Legacy Code:
f = open("data1.txt")
print(f.read())
f.close()

f = open("data2.txt")
print(f.read())
f.close()

Code:

```
def read_and_print(filename):
    try:
        with open(filename) as f:
            print(f.read())
    except FileNotFoundError:
        print(f"File '{filename}' not found.")
read_and_print("data1.txt")
read_and_print("data2.txt")
```

```
def read_and_print(filename):
    try:
        with open(filename) as f:
            print(f.read())
    except FileNotFoundError:
        print(f"File '{filename}' not found.")

read_and_print("data1.txt")
read_and_print("data2.txt")
```

```
File 'data1.txt' not found.
File 'data2.txt' not found.
```

Observation:

The legacy code repeatedly opened, read, and closed files, which violated the DRY (Don't Repeat Yourself) principle. Refactoring introduced a reusable function that accepts the filename as a parameter. The with open() context manager was used to automatically handle file closing. This made the code cleaner, safer, and easier to reuse for multiple files. It also reduced the chances of resource leaks caused by forgetting to close files.

4. Optimizing Search Logic

• Task: Refactor inefficient linear searches using appropriate data structures.

Prompt:

Refactor the following Python code to optimize the search operation. Replace the linear search with a more efficient data structure such as a set or dictionary. Explain the improvement in time complexity.

Legacy Code:
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied**")**

Code:

users = {"admin", "guest", "editor", "viewer"}

name = input("Enter username: ")

print("Access Granted" if name in users else "Access Denied")

```
# Optimized search using a set for O(1) lookup (task4)
users = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")
print("Access Granted" if name in users else "Access Denied")
```

```
Enter username: Anshu
Access Denied
```

```
Enter username: guest
Access Granted
```

Observation:

The original program searched for a username in a list using a loop, which resulted in linear time complexity O(n). This approach becomes inefficient when the number of users increases. The refactored code replaced the list with a set, which supports faster membership checking. Sets provide average lookup time of O(1), making the search operation much more efficient. This change improved both performance and code simplicity.

Task 5 – Refactoring Procedural Code into OOP Design

Prompt:

Refactor the following procedural Python code into an Object-Oriented design. Create a class called EmployeeSalaryCalculator with attributes and methods to calculate tax and net salary.

Legacy Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Code:

```
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary
    def calculate_tax(self):
        return self.salary * 0.2
    def calculate_net_salary(self):
        return self.salary - self.calculate_tax()
calculator = EmployeeSalaryCalculator(50000)
print(calculator.calculate_net_salary())
```

```
#task5
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        return self.salary * 0.2

    def calculate_net_salary(self):
        return self.salary - self.calculate_tax()

calculator = EmployeeSalaryCalculator(50000)
print(calculator.calculate_net_salary())
```

```
40000.0
```

Observation:

The legacy code followed a procedural approach where salary calculations were performed directly using variables. Refactoring transformed this into a class-based design called EmployeeSalaryCalculator. The class encapsulates salary, tax calculation, and net salary methods within a single structure. This improves organization and promotes the use of object-oriented programming principles. It also makes the code more scalable and reusable for future extensions.

Task 6 – Refactoring for Performance Optimization

Prompt:

Refactor the following Python code to improve performance. Replace the loop with a more efficient method such as a mathematical formula or built-in function while keeping the same result.

Legacy Code:
total = 0
for i in range(1, 1000000):
if i % 2 == 0:
total += i
print(total)

Code:

```
# Efficient sum of even numbers from 1 to 999999 using arithmetic progression formula
# First even number: 2, Last even number <= 999999: 999998, Number of terms: 499999
n = (999998 - 2) // 2 + 1
total = n * (2 + 999998) // 2
print(total)
```

```
249999500000
```

Observation:

The original code calculated the sum of even numbers using a loop that iterated up to one million. This approach required many iterations and increased execution time. The refactored solution used a mathematical formula to compute the sum directly. By replacing the loop with a constant-time calculation, the time complexity was reduced significantly. This optimization improves performance, especially when dealing with large datasets.

Task 7 – Removing Hidden Side Effects

Prompt:

Refactor the following Python code to remove hidden side effects caused by modifying a global list. Rewrite the function so it returns a new value instead of mutating shared state.

Legacy Code:
data = []
def add_item(x):
data.append(x)

add_item(10)
add_item(20)
print(data)

Code:

```
def add_item(data, x):
   return data + [x]

data = []

data = add_item(data, 10)

data = add_item(data, 20)

print(data)
```

```
#task7
def add_item(data, x):
    return data + [x]

data = []
data = add_item(data, 10)
data = add_item(data, 20)
print(data)
```

```
[10, 20]
```

Observation:

The original function modified a global list directly by appending values to it. This created hidden side effects because the function changed shared data outside its scope. In the refactored version, the function returns a new list instead of modifying the global variable. This makes the function more predictable and easier to test. Functional-style programming practices help reduce unexpected behavior in larger programs.

Task 8 – Refactoring Complex Input Validation Logic

Prompt:

Refactor the following Python code to simplify the password validation logic. Break the validation into separate functions for readability and testability while maintaining the same functionality.

Legacy Code:
password = input("Enter password: ")
if len(password) >= 8:

```
if any(c.isdigit() for c in password):
if any(c.isupper() for c in password):
print("Valid Password")
else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
```

Code:

```
def is_long_enough(password):
    return len(password) >= 8
def has_digit(password):
    return any(c.isdigit() for c in password)
def has_uppercase(password):
    return any(c.isupper() for c in password)
def validate_password(password):
    if not is_long_enough(password):
        print("Password too short")
    elif not has_digit(password):
        print("Must contain digit")
    elif not has_uppercase(password):
        print("Must contain uppercase")
    else:
        print("Valid Password")
password = input("Enter password: ")
validate_password(password)
```

```python
def is_long_enough(password):
    return len(password) >= 8
def has_digit(password):
    return any(c.isdigit() for c in password)
def has_uppercase(password):
    return any(c.isupper() for c in password)
def validate_password(password):
    if not is_long_enough(password):
        print("Password too short")
    elif not has_digit(password):
        print("Must contain digit")
    elif not has_uppercase(password):
        print("Must contain uppercase")
    else:
        print("Valid Password")
password = input("Enter password: ")
validate_password(password)
```

```
Enter password: Anshu@123
Valid Password
```

```
Enter password: anshu
Password too short
```

Observation:

The original password validation code contained multiple nested conditions, which made it difficult to understand and maintain. During refactoring, the validation rules were separated into individual functions such as checking length, digits, and uppercase letters. This modular approach improved readability and allowed each rule to be tested independently. The final structure made the code cleaner and easier to extend with additional validation rules if required.